4. 실험

4.1 실험 설정

4.1.1 자료와 워크로드

8개 리전의 2025년 시간별 탄소집약도는 Electricity Maps[31]에서 받은 소비 기준 평균 배출계수다[32]. 리전별 구역은 US-CAL-CISO(캘리포니아), US-TEX-ERCO(텍사스), US-NY-NYIS(뉴욕), FR, DE, KR, JP, IN이며, 일본과 인도는 나라 단위 구역이다. 3.2절의 LSTM은 2021~2023년 자료로 학습하고 2024년 자료로 검증했다. Adam(learning rate 0.001)과 batch size 64로 학습했고, 검증 오차가 더 줄지 않으면 early stopping을 적용했다(20~150 epoch). 세 리전(캘리포니아, 텍사스, 독일)이 받는 예측 구간의 기상값은 실측값이다. 예측은 과거 168시간이 쌓이는 1월 8일부터 나오므로, 작업은 1년치를 시뮬레이션하되 1월 8일 이후에 제출된 작업만 평가한다(평가 기간 8,592시간).

워크로드는 두 가지이며 표 4에 비교했다. 표의 실행 시간은 1년 전체 작업 기준이다. 합성 워크로드는 리전마다 하루 50건씩 만든 작업이다. 작업은 허용 대기 시간에 따라 다섯 등급으로 나누고(표 5), 실행 시간은 등급마다 로그정규 분포에 상·하한을 두어 뽑는다. 표 5의 실행 시간은 중앙값이고 괄호는 합성 워크로드의 상·하한이다. 도착 시각은 그림 4와 같다. 등급 1~2는 현지 2시, 등급 3~5는 현지 13시를 중심으로 표준편차 3시간의 정규분포에서 70%가 도착하고, 나머지 30%는 그 반대쪽 12시간에 고르게 도착한다. 현지 시각은 일광절약시간 없이 고정 시차로 둔다. 허용 대기 시간이 짧은 등급 3~5의 작업은 대부분 연기 불가 작업이 된다.

Azure 워크로드는 Azure의 2019년 VM 기록[34, 35]에서 만들었다. 30일 기록에 담긴 VM 약 270만 개 가운데 기록의 처음과 끝에서 잘린 VM을 빼고 146,016개를 뽑아 1년으로 이었다. 도착 시각과 실행 시간은 기록 그대로이고, 제출 리전은 여덟 리전에 고르게 나눈다. 등급은 합성과 같은 비율로 무작위로 주므로 등급마다 실행 시간이 같다. 두 워크로드는 작업 수가 같지만, Azure는 작업이 훨씬 길어 연기 불가 작업이 실행 시간의 절반을 차지하고 도착은 하루 내내 고르다.

4.1.2 상한, 예측기, 비교 대상

리전 상한은 3장 표 3의 규칙대로 기준 피크의 1.25배를 올림해 모든 리전에 같게 주며, 합성 17, Azure 43이다. 예비 용량은 첫 주에 연기 불가 작업이 한 리전에서 동시에 실행된 최대 수로 정해 합성은 3이다. Azure는 연기 불가 VM이 여러 날 실행되어 이 최대 수의 대부분이 이미 실행 중인 VM이므로, 같은 VM을 실행 수와 예비 용량에서 두 번 세지 않도록 0으로 둔다. 예측기는 3.2절의 LSTM을 기본으로 쓰고, 24시간 전 같은 시각의 값을 쓰는 예측과 실측값을 그대로 쓰는 완전예지를 함께 쓴다.

비교 대상은 2장의 선행 연구 가운데 리전과 시각을 함께 옮기는 정책이다(표 6). Sukprasert 등[6]의 정책은 1년치 실측 탄소집약도를 미리 안다고 보고, 공개된 탐색 코드를 1시간 단위로 실행해 작업별 마감 안에서 가장 깨끗한 리전과 시각을 찾는다. 용량은 두지 않는다. Caspian[8]은 원문의 선형 계획을 구현하되, 1시간 미만 작업은 실행 시간만큼만 슬롯 자원을 쓰게 하고 탄소는 LSTM 예측을 썼다. 용량은 하드 제약으로, 마감은 페널티로 두며, 마감을 1시간 넘긴 작업은 제출 리전에서

표 4 두 워크로드

Table 4 The two workloads

Item	Synthetic	Azure
Jobs evaluated	143,200	143,165
Median run (sec)	81	1,500
95th-pct run (sec)	13,074	74,400
Non-deferrable run time (%)	1.3	51
Baseline peak	13	34
Ceiling	17	43
Reserve	3	0
Baseline emissions (kg)	28,625	213,513

표 5 등급별 허용 대기 시간과 실행 시간

Table 5 Allowed wait and run time by class

Class	Allowed wait	Synthetic run (sec)	Azure run (sec)	Share (%)
1	12–24 h	5,400 (300–21,600)	1,500	30
2	2–8 h	1,800 (60–14,400)	1,500	10
3	2–10 min	180 (5–1,800)	1,500	10
4	15–60 s	30 (1–300)	1,500	20
5	1–5 s	5 (1–60)	1,500	30

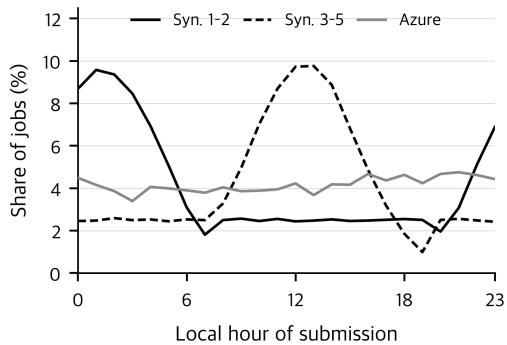


그림 4 현지 시각별 도착 비율

Fig. 4 Share of arrivals by local hour

표 6 비교 대상

Table 6 Compared policies

Policy	Capacity	Deadline	Re-solve	Carbon
Sukprasert	None	Hard	Offline	Actual
Caspian	Hard	Soft	Hourly	LSTM
Arrival-time	Hard	Hard	No	LSTM
CAST	Soft	Hard	Hourly	LSTM

표 7 평가 지표

Table 7 Metrics

Metric	Definition
Savings (%)	Emission reduction against running every job at once in its home region
Violations	Jobs started in a region already at its ceiling
RTT (ms)	Mean round-trip latency, weighted by run time
Wait (hr)	Mean time from submission to start
Req. cap.	Ceiling a policy needs to run without violations

바로 시작하게 했다. 도착 시 배정 방식[9]은 작업이 도착한 슬롯에서 이미 정한 모든 계획과 함께 상한을 확인해 리전과 시작 시각을 한 번 정한 뒤 다시 풀지 않는다. 이 방식은 CAST와 같은 선형 계획, 예측, 비용 위에 구현해 확정 시점만 다르게 했다. 모든 정책은 같은 자료와 배출 계산 방식으로 채점한다.

4.1.3 지표

지표는 표 7과 같다. CAST와 다른 방식의 배출을 같은 지연에서 비교할 때는 한쪽의 고정 α를 이분 탐색으로 조정해 두 방식의 왕복 지연 시간 차이를 0.6 ms 안으로 맞춘 뒤 비교한다. 95% 신뢰구간은 7일 단위로 묶어 2,000번 다시 뽑는 블록 부트스트랩으로 구한다.

4.2 주요 결과

4.2.1 탄소 절감과 지연

CAST는 두 워크로드 모두 상한 위반 없이 배출을 줄였다(표 8). 무릎점 규칙은 사람이 α를 정하지 않고 슬롯마다 골라 합성에서 71.93%, Azure에서 51.35%를 줄였고, 탄소만 고려하는 α=1에서는 각각 95.65%, 67.74%를 줄였다.

시간 이동만은 리전을 바꾸지 않고 시작 시각만 옮긴 실행이고, 리전 이동만은 시작 시각을 바꾸지 않고 리전만 옮긴 실행이다. 합성에서 둘은 각각 18.78%와 61.57%를 줄인다. 리전 이동만은 왕복 지연 시간이 CAST와 거의 같지만 CAST가 10.36%p 더 줄인다. Azure는 연기 불가 작업이 실행 시간의 절반이라 시간 이동만으로는 1.53%를 줄이며, 리전 이동만이 52.10%로 CAST보다 0.75%p 높고 왕복 지연 시간은 약 7 ms 더 길다.

계절이 다른 네 주에서는 CAST의 배출을 하한과 비교했다. 하한은 그 주의 탄소집약도와 작업을 모두 안다고 보고, 상한 17 안에서 한 주 전체를 한 번에 푼 배정의 배출이다. α=1의 CAST는 이 하한과 0.33~1.47%p 차이다.

표 8 주요 결과

Table 8 Main results

Setting	Synthetic (%)	RTT (ms)	Azure (%)	RTT (ms)
CAST, knee rule	71.93	47.5	51.35	39.4
CAST, α=1	95.65	121.3	67.74	115.8
Time only	18.78	0	1.53	0
Region only	61.57	47.3	52.10	46.5

4.2.2 가중치 α에 따른 배정

α가 커질수록 작업이 깨끗한 리전으로 옮겨 가고, 절감과 왕복 지연 시간이 함께 커진다(표 9). 표의 FR, CA, Moved는 프랑스, 캘리포니아에서 실행된 몫과 옮긴 작업의 몫이며, 모두 실행 시간 기준이다. α=0.4까지는 옮긴 작업이 24% 이하이고 절감은 38.64% 이하다. α=0.5~0.7에서 작업이 프랑스와 캘리포니아로 옮겨 가며 절감이 63%에서 89%로 오르고, α=1에서는 실행 시간의 99.9%가 프랑스에서 실행되며 왕복 지연 시간이 121 ms가 된다.

무릎점 규칙은 슬롯의 89%에서 0.5 또는 0.6을 고르고, 고른 α의 평균은 시간대에 따라 0.52~0.61로 조금 움직인다. 대기 시간은 1.47시간으로 α=0을 뺀 고정 α 열 개보다 짧다.

왕복 지연 시간이 같은 고정 α(0.575)는 73.79%로 무릎점 규칙보다 1.87%p 더 줄이지만, 대기 시간은 1.63시간으로 길다. 합성 아홉 조건(예측기 셋, 상한 10과 7, 작업을 다시 뽑은 네 번) 가운데 여덟에서 같은 지연의 고정 α가 1.77~2.64%p 더 줄였고, Azure에서는 차이가 없었다(0.63%p, 신뢰구간이 0을 포함). 대기 시간은 모든 조건에서 무릎점 규칙이 짧았다. 이 고정 α는 1년을 실행한 뒤에야 알 수 있는 값이다. 첫 주 자료로 정한 고정 α(0.5687)는 72.86%로, 무릎점 규칙과 0.93%p 차이다.

표 9 α에 따른 배정(합성)

Table 9 Assignment by α (synthetic)

α	Savings (%)	RTT (ms)	Wait (hr)	FR	CA	Moved
0.0	0.00	0.0	0.00	12.3	12.7	0.0
0.1	23.48	0.5	1.89	16.8	12.7	4.5
0.2	26.61	1.2	1.90	22.5	12.7	10.2
0.3	29.25	2.3	1.95	23.9	15.0	14.0
0.4	38.64	9.1	1.92	27.7	20.0	24.3
0.5	62.99	32.0	1.73	43.2	24.7	47.5
0.6	77.69	53.7	1.58	53.4	32.9	64.7
0.7	88.99	78.7	1.52	65.2	33.6	75.4
0.8	93.51	98.2	1.74	81.1	18.9	80.6
0.9	95.24	113.7	2.11	93.9	6.1	85.1
1.0	95.65	121.3	2.42	99.9	0.1	87.7
Knee	71.93	47.5	1.47	49.2	31.2	59.0

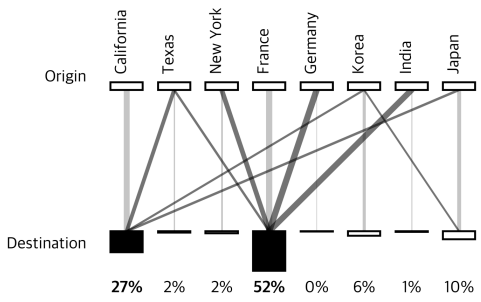


그림 5 리전 간 작업 이동(합성, 무릎점, 건수 기준)

Fig. 5 Job flows between regions (synthetic, knee, by job count)

4.2.3 리전 간 이동

그림 5는 무릎점 규칙에서 작업이 옮겨 간 흐름이다. 실행 시간 기준으로는 49%가 프랑스, 31%가 캘리포니아에서 실행된다(표 9). 독일과 인도의 작업은 주로 프랑스로, 텍사스, 일본, 한국의 작업은 주로 캘리포니아로 간다.

4.3 선행 정책과의 비교

4.3.1 용량이나 마감을 양보하는 정책

두 선행 정책은 용량과 마감 가운데 하나를 양보한다(표 10, 그림 6). Sukprasert의 정책은 용량을 두지 않고, Caspian은 마감을 페널티로 둔다. CAST는 마감을 하드 제약으로, 용량을 소프트 제약으로 두어 이 실험에서 상한 위반과 마감 위반이 모두 없었다. 용량을 두지 않는 정책은 상한 17에서의 위반과 함께 필요 용량으로 비교한다.

Sukprasert의 정책은 96.63%를 줄이지만, 그 배정이 위반 없이 실행되려면 한 리전이 동시에 129개를 실행해야 한다. 기준 피크 13의 9.9배이며, 작업의 99.75%를 프랑스에서 실행한다. CAST(α=1)는 1년치 실측 대신 24시간 예측으로, 상한 17 안에서 95.65%를 줄여 그 차이가 0.98%p다.

표 10 선행 정책과의 비교(합성)

Table 10 Comparison with prior policies (synthetic)

Policy	Savings (%)	Viol.	Req. cap.	Deadline miss	RTT (ms)
Sukprasert	96.63	61,539	129	0	121.3
Caspian	67.54	1,538	34	5,582	84.5
CAST	71.93	0	17	0	47.5
CAST, α=1	95.65	0	17	0	121.3

Caspian은 67.54%를 줄이며, 마감을 페널티로 두어 5,582건(3.90%)이 마감을 넘긴다. Caspian은 용량을 슬롯 안 실행 시간의 합으로 확인하므로, 배정을 동시 실행수로 다시 세면 1,538건이 상한 17을 넘고 필요 용량은 34(기준 피크의 2.6배)다.

그림 6의 (a)는 필요 용량에 따른 절감, (b)는 왕복 지연 시간에 따른 절감이다. (a)에서 CAST의 두 곡선은 상한을 7~39로 바꾼 결과이고, 점선은 상한이 없을 때의 CAST다. (b)에서 CAST의 곡선은 α를 고정한 결과다. CAST 무릎점은 Caspian보다 왕복 지연 시간이 짧으면서(47.5 대 84.5 ms) 더 많이 줄이고, Sukprasert는 CAST 곡선의 끝인 α=1과 같은 지연에서 비슷하게 줄인다.

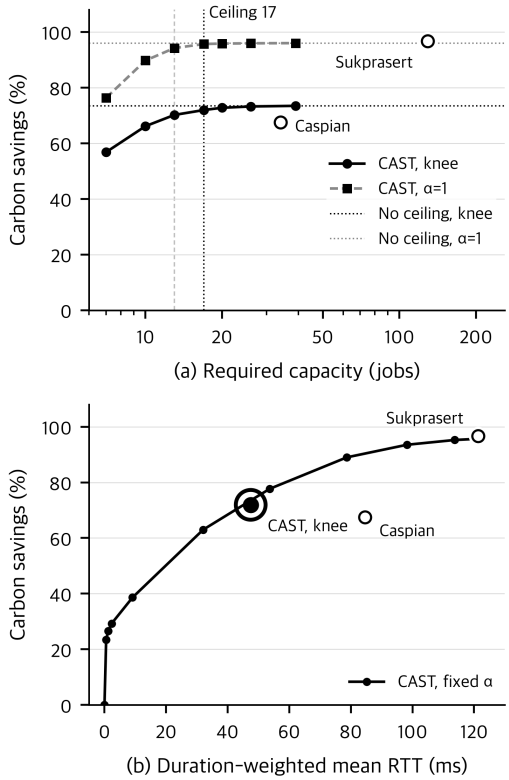


그림 6 선행 정책과의 비교: (a) 필요 용량, (b) 왕복 지연 시간

Fig. 6 Comparison with prior policies: (a) required capacity, (b) round-trip latency

표 11 도착 시 배정 방식 대비 CAST의 배출 차(%p)

Table 11 Emission gap between the arrival-time method and CAST (pp)

Condition	Gap (pp)	95% CI
Synthetic, cap 17	0.96	0.51 to 1.46
Synthetic, 4 more draws	1.23 to 1.56	all above 0
Synthetic, cap 13	1.46	1.13 to 1.83
Synthetic, cap 26	0.34	−0.06 to 0.77
Synthetic, small caps in clean regions	2.10	1.83 to 2.36
Synthetic, oracle	1.01	0.80 to 1.25
Synthetic, no future information	0.96	0.74 to 1.18
Azure, cap 43	2.53	1.68 to 3.42

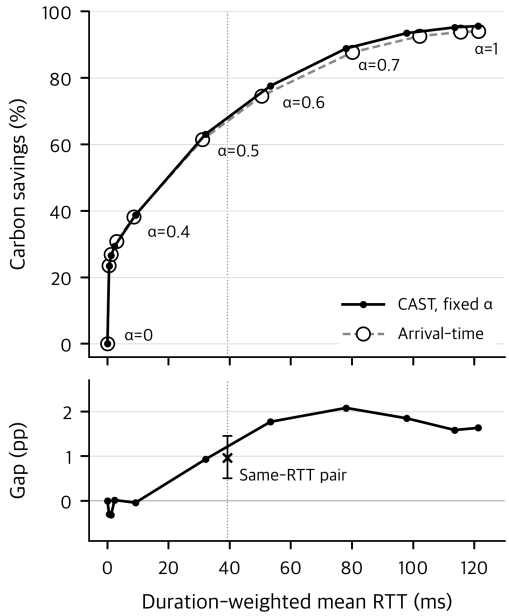


그림 7 α를 고정한 CAST와 도착 시 배정 방식

Fig. 7 CAST and the arrival-time method with fixed α

4.3.2 도착 시 배정 방식

작업이 시작할 때 확정하는 CAST는 도착할 때 확정하는 방식보다 같은 지연에서 덜 배출한다. 두 방식은 같은 선형 계획, 예측, 비용을 쓰고 확정 시점만 다르며, 둘 다 상한 위반이 없다. 확정 시점만 비교하도록 이 절에서는 두 방식 모두 예비 용량을 두지 않는다.

표 11은 지연을 맞춘 두 방식의 배출 차이며, 양수는 CAST가 덜 배출한 양이다. 왕복 지연 시간을 같게 맞추면 CAST가 기준 배출 대비 합성에서 0.96%p, Azure에서 2.53%p 덜 배출한다. 확정을 시작 시점까지 미루므로, 그사이 도착한 작업과 함께 자리를 다시 나눌 수 있기 때문이다. 작업을 다시 뽑거나 예측기와 미래 정보를 바꾸어도 차이는 0보다 크다. 차이는 깨끗한 리전의 상한이 작을수록 커서, 깨끗한 세 리전(캘리포니아, 프랑스, 뉴욕)의 상한을 작게 두면 2.10%p이고, 상한이 넉넉한 26에서는 0.34%p로 신뢰구간이 0을 포함한다.

이 차이는 α 전체에서 나타난다. 그림 7은 α를 0에서 1까지 고정해 두 방식을 실행한 곡선이며, 점은 왼쪽부터 α=0, 0.1, …, 1이고 아래 칸은 같은 왕복 지연 시간에서 두 곡선의 차이다. 왕복 지연 시간이 10 ms 아래인 $\alpha \leq 0.4$에서는 두 곡선이 겹치고, 30 ms 위에서는 같은 지연에서 CAST가 0.9~2.1%p 위에 있다.

두 방식에 상한 위반이 없는 것은 모든 대기 작업의 계획을 함께 세기 때문이다. 도착 시 배정 방식이 앞서 배정한 작업의 계획을 무시하고 실행 중인 작업만 세도록 바꾸면, 합성의 다섯 워크로드에서 작업의 9.0~17.0%,

표 12 조건별 결과

Table 12 Results under changed conditions

Condition	Knee (%)	α=1 (%)	Viol.
Base setting	71.93	95.65	0
Base setting, reserve 0	72.06	95.62	0
SN-24h	64.44	95.59	0
Oracle	67.20	95.87	0
5 workload draws	71.51 ± 0.43	95.40 ± 0.28	0
Cap 7 (0.5×)	56.79	76.30	0
Cap 13 (1×)	70.16	94.18	0
Cap 26 (2×)	73.21	95.90	0
Cap 39 (3×)	73.42	95.96	0
Per-region caps (3)	63.75–73.05	87.38–95.77	0
Arrivals ×2	69.45	92.58	0
Class 1–2 peak at 13 h	70.26	95.58	0
Deferrable 20–60%	70.01–72.54	93.35–95.70	0
Horizon 6–12 h	71.04–71.63	95.58–95.65	0
No future info, reserve 0	68.88	—	0
RTT cap 50 ms	28.41	38.09	0
RTT cap 100 ms	32.98	46.83	0
RTT cap 150 ms	66.37	89.85	0
Azure, cap 0.5×	19.70	—	465
Azure, cap 0.6×	27.17	—	0
Azure, cap 0.75–2×	34.15–61.89	—	0

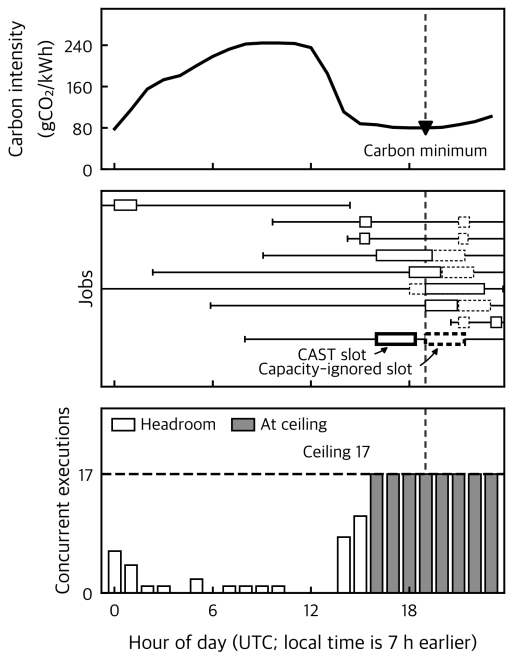


그림 8 캘리포니아의 하루(7월 7일)

Fig. 8 One day in California (7 July)

Azure에서 11.4~14.7%가 상한을 넘겨 시작한다. 같은 워크로드에서 CAST의 위반은 0이다.

4.4 조건별 결과

4.4.1 상한과 부하

합성에서는 상한을 바꾸어도 CAST의 위반이 0이다(표 12). 상한을 늘리면 절감이 오르다가 기준 피크의 2배(26) 근처부터 거의 늘지 않고, 상한이 없을 때의 무릎점 절감(73.39%)에 다가간다. 상한 17에서 용량을 지키느라 덜 줄인 양은 1.47%p다. 리전마다 상한을 달리 나누어도 위반은 없었다. Azure에서는 상한을 기준 피크의 0.5배(17)로 줄였을 때 465건(0.32%)이 상한을 넘겼고, 모두 모든 리전이 상한까지 찬 순간이었다. 같은 상한에서 도착 시 배정 방식은 847건(0.59%)이다.

그림 8은 상한이 실제로 걸리는 모습을 캘리포니아의 7월 7일로 보인다. 가운데 칸의 실선은 CAST가 정한 실행 구간, 파선은 상한이 없을 때의 실행 구간이다. 탄소가 낮은 현지 9~16시(UTC 16~23시)에 CAST의 동시 실행이 상한 17에 이르고, 이 시간에 들어가지 못한 작업은 그 앞뒤 시각에 시작한다.

4.4.2 예측기, 작업, 미래 정보

예측기, 작업, 도착을 바꾸어도 위반은 0이다(표 12). 탄소만 고려하는 α=1에서 절감은 세 예측기(LSTM, 24시간 전 값인 SN-24h, 완전예지인 Oracle) 사이에서 0.3%p 안에 있다. LSTM의 평균 절대 오차는 26.10 gCO_2/kWh로, 직전 시각의 값(48.41)이나 168시간 전 값(35.46)보다 작고 24시간 전 값(25.27)과 비슷하다. 무릎점 규칙에서는 예측기마다 고르는 α가 달라 절감과 지연이 함께 움직이며, 완전예지의 무릎점 절감(67.20%)이 LSTM(71.93%)보다 낮은 것도 이 때문이다. 두 실행이 슬롯마다 고른 α를 서로 바꾸어 주고 평균하면, 이 차이의 약 72%가 고른 α에서 온다.

작업을 다시 뽑거나 연기 가능 작업의 비율, 등급 1~2의 도착 중심(현지 2시에서 13시로), 하루 도착 수, 예측 구간을 바꾸어도 무릎점 절감은 69.45~72.54%다. 미래 정보(슬롯 안 도착, 이번 슬롯의 실측 탄소집약도, 예측 구간의 실측 기상값)를 모두 빼면 예비 용량이 없는 기본 설정보다 3.18%p 준 68.88%다.

4.4.3 왕복 지연 상한

주 실험은 지연 상한이 없어 모든 작업이 어느 리전으로든 옮겨 갈 수 있으므로 리전 이동에 유리하다. 모든 작업에 50 ms나 100 ms의 상한을 두면 한국, 일본, 인도의 작업이 캘리포니아와 프랑스에 닿지 못해 절감이 28.41%, 32.98%로 준다. 150 ms에서는 66.37%다.

표 13 배출 계산 방식별 배출 차(%p)

Table 13 Emission gap by emission accounting (pp)

Accounting	Synthetic	Azure
Average factor	0.96	2.53
Exact overlap	0.88	2.52
Transfer 1 GB, destination	1.03	2.50
Transfer 1 GB, origin	1.01	2.59
Life-cycle approx.	0.95	2.40
PUE 1.6 at FR, CA	1.07	2.32
PUE 1.1 at FR, CA	0.89	2.68
Marginal, France only	−0.89	1.48
Marginal, symmetric	1.19	2.05
France only + origin 1 GB + PUE 1.6	−1.49	0.89

4.4.4 계산 비용

CAST는 매 슬롯 후보 α 11개마다 선형 계획 문제를 푼다. SciPy 1.18.0의 HiGHS로 Apple M1 Max(메모리 32 GB)에서 실행했을 때, 합성의 슬롯당 풀이 시간은 평균 223 ms, 최대 1,223 ms로 슬롯 길이 1시간보다 훨씬 짧다. 한 슬롯 문제의 변수는 가장 많을 때 7,848개다. Azure는 슬롯당 평균 318 ms이고, 합성의 도착을 두 배로 늘리면 변수가 15,816개, 슬롯당 평균 556 ms다.

4.5 배출 계산 방식

배출을 다르게 세어도 대부분 CAST가 도착 시 배정 방식보다 덜 배출한다. 표 13은 표 11의 첫 비교를 배정은 그대로 두고 배출만 다시 센 결과이며, 양수는 CAST가 덜 배출한 양이다. 실행이 슬롯과 겹친 길이로 정확히 세거나, 옮긴 작업마다 1 GB의 전송 에너지(0.06 kWh[37])를 더하거나, 무탄소 전원에 전과정 배출(원자력 12, 재생 30 gCO_2/kWh)을 더하거나, 리전 설비 효율(PUE)을 달리 두어도 차이는 합성 0.88~1.07%p, Azure 2.32~2.68%p다.

순서가 뒤집히는 것은 합성의 두 경우다. 한계 배출을 근사하려고 프랑스로 옮긴 작업을 같은 시각 독일의 탄소집약도로 세면 도착 시 배정 방식이 0.89%p 덜 배출하고, 여기에 출발 리전으로 센 1 GB 전송과 프랑스, 캘리포니아의 PUE 1.6을 더하면 1.49%p 덜 배출한다. Azure에서는 두 경우 모두 CAST가 덜 배출한다. 모든 리전의 탄소집약도를 화석 전원의 몫(최소 0.05)으로 나누어 세는 대칭 근사에서는 CAST가 합성 1.19%p, Azure 2.05%p 덜 배출한다.

전송 에너지를 출발 리전으로 세면, 옮긴 작업마다 약 9 GB를 보낼 때 CAST 무릎점의 절감이 작업을 옮기지 않는 시간 이동만(18.78%)과 같아진다. Azure VM의 코어 수에 비례하게 작업 전력을 주고 다시 세어도 절감은 많아야 1.1%p 움직이고 비교 순서는 같다.